# Intuitive Understanding of Attention

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

- **What do Q, K, and V mean, and how do they work in attention computation?**
  - Let's see how the equivalent counterparts work in python:

```python
# a python dictionary with keys and values
dictionary = {"a": 1,
              "b": 2,
              "c": 3,
              "d": 100}

# define a query
query = 'd'

# check if query is persent in the dictionary
dictionary[query]
# 100
```

  - Here we can regard the *query* variable as Q, keys of the dict as K, and values of the dict as V. Then let it become more complicated...

```python
# now, keys and values are vectors
dictionary = {
    ( 2.3,  0.4, -1.9): [ 1.6, -0.1,  1.8],
    ( 1.5, -0.8,  4.2): [-0.4,  0.5, -0.8],
    (-0.3,  2.8,  5.5): [ 0.0, -0.8,  0.7],
    ( 2.6,  1.8,  2.4): [-0.7,  1.5,  1.2]
}

# define a query vector
query = (1.5, -0.8, 4.2)

# look for the query vector in the dictionary
dictionary[query]
# [-0.4,  0.5, -0.8]
```

  - Now Q, K, and V are vectors. However, it is worth noting that the dictionary-query in python is a **HARD QUERY**. When there isn't the queried key, then it will be an error.
  - In transformer, the **ATTENTION** mechanism is something similar to the Q, K, and V's in python dict, but instead of using HARD QUERY, it uses **SOFT QUERY**, where the query (Q) is compared with all available keys (K) to compute **similarities**. Instead of exact matching, it produces a probability distribution (using softmax) over all keys, indicating how much attention to give to each one.
- **Math basics: dot product can reflect "similarities".**
  - Two vectors pointing in the same or similar directions have a positive dot product. A larger

positive value indicates they are more closely aligned (i.e., more similar in direction and proportion).

- A dot product of zero means the vectors are orthogonal (perpendicular), showing no directional overlap.
- A negative dot product indicates the vectors point in mostly opposite directions, signifying "negative alignment" or dissimilarity in direction.
    - *Examples*
        - Two same vector:

$$\mathbf{a} = [1, 2], \quad \mathbf{b} = [1, 2]$$

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 1 + 2 \cdot 2 = 1 + 4 = 5$$

        - Opposite direction:

$$\mathbf{a} = [1, 2], \quad \mathbf{b} = [-1, -2]$$

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot (-1) + 2 \cdot (-2) = -1 - 4 = -5$$

        - Perpendicular:

$$\mathbf{a} = [1, 0], \quad \mathbf{b} = [0, 1]$$

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 0 + 0 \cdot 1 = 0$$

- Somewhat similar:

$$\mathbf{a} = [1, 2], \quad \mathbf{b} = [2, 1]$$

$$\mathbf{a} \cdot \mathbf{b} = 1 \cdot 2 + 2 \cdot 1 = 2 + 2 = 4$$

- In python

```python
keys = torch.tensor(
    [
        [ 2.3,  0.4, -1.9],
        [ 1.5, -0.8,  4.2],
        [-0.3,  2.8,  5.5],
        [-1.1,  2.5,  4.4],
    ]
) # shape [4, 3]

query = torch.tensor([[-0.5, 2.9, 5.4]])  # shape [1, 3]

dim = query.shape[-1]
scores = torch.mm(query, keys.T) / math.sqrt(dim)
print("scores:", scores)
# scores:
# tensor([[-5.9178, 11.3218, 21.9220, 18.2212]])
```

It can be seen that scores[2] has the max value as the similarity between the query and the key[2] is the highest.

- Sometimes we may encounter situations like:

$$A = [0, 1]$$
$$B = [1000, 1000]$$
$$C = [0, 1]$$

We may find that the dot product of A and B is much greater than that of A and C, despite that A and C are completely the same. In this case, we may normalize (or use other scale methods) the Q, K, and V to eliminate this effect.

- **Back to Attention...**
  - The $QK^\top$ is the process of finding the similarities between the "querier" token and the "queriee" tokens. Here the term "similarity" is just an abstract concept, which will be discussed later.
    - *Example (literal meaning of similarity):*

```
"came" and "come" may have a larger product, say, 10, while "came" and "shoe" may have a  significantly small product, say 1.
```

- To avoid producing an absurdly large number, we use $\frac{1}{\sqrt{d_k}}$ as a scale factor, and then use $\mathrm{softmax}(\cdot)$ to convert the product to a probability.

```python
# (from python example above)
weights = torch.softmax(scores, dim=1)
print("weights:", weights)
# weights: tensor([[7.9196e-13, 2.4311e-05, 9.7587e-01, 2.4107e-02]])
```

- To query the value, which is the last part of equation (1), we multiply the softmaxed vector with the V vector.

```python
values = torch.tensor(
    [
        [ 1.6, -0.1,  1.8],
        [-0.4,  0.5, -0.8],
        [ 0.0, -0.8,  0.7],
        [-0.7,  1.5,  1.2]]
)

attn = torch.mm(weights, values)
print("attn:", attn)
# attn: tensor([[-0.0169, -0.7445,  0.7120]])
```

From here we can see that the result attention is closest to values[2], which corresponds to what we saw above.

- **But, what does the similarity mean here anyway?**
  - This is the most elegant part of the multihead attention mechanism. Each head is responsible for a semantic space. Intuitively (but not precisely), the term "semantic space" can be regarded as the grammar relationship in linguistics (S-V, V-O, S-O, S-V-O...).
  - For the head of the grammar function, if the querier token and the queriee token satisfy the corresponding function, then we can say they are similar.
    - *For example:*

```
Given a head, h, which is responsible for finding S-V relationship.
The input is: I eat apples, where "I" is the subject and "eat" is the verb.
Then "I" and "eat" (also "eat" and "I") are (much) more "similar" than other pairs. Thus, the weight is larger, and "I" will pay more attention on "eat" and vice versa.
```

- **Work cited and more examples with the vision transformer:**
  - [An Intuitive Introduction to the Vision Transformer]
  - [Chat-GPT discussion]